

Smart Hostel Management System



NAMAL UNIVERSITY MIANWALI

Project Milestone 3 Report

By

	Name	Roll No.
1	Muhammad Ahmad	NUM-BSCS-2024-44
2	Asad Ullah Khan	NUM-BSCS-2024-17
3	Maryam Rashid	NUM-BSCS-2024-33

CSC-271 Database Systems
Department of Computer Science
Namal University, Mianwali

Submission Date: 13 June 2026

TABLE OF CONTENTS

CHAPTER 1: PROJECT OVERVIEW	3
1.1. INTRODUCTION:	3
1.2. PROBLEM STATEMENT:.....	3
1.3. PROJECT OBJECTIVES:	4
CHAPTER 2: CONCEPTUAL DATABASE DESIGN.....	5
2.1. ENTITY:	5
2.2. DATA DICTIONARY:	6
2.3. BUSINESS RULES:	12
2.4. ENTITY RELATIONSHIP DIAGRAM:	13
CHAPTER 3: LOGICAL DATABASE DESIGN.....	14
3.1. RELATIONAL SCHEMA:	14
3.2. FUNCTIONAL DEPENDENCIES:	15
3.3. NORMALIZATION:	16
CHAPTER 4: IMPLEMENTATION	16
4.1. LANGUAGE/Framework:	19
4.2. DATABASE CONNECTIVITY:	19
4.3. STORED PROCEDURES AND FUNCTIONS:.....	19
4.4. INTERFACES:	20
REFERENCES	26
APPENDIX: AI USAGE AND PROMPTS	27

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION:

The Smart Hostel Management System (SHMS) is a centralized, database-driven application designed to digitize the daily administrative operations of the female hostel at Namal University, Mianwali. The hostel currently accommodates students from four degree programs BSCS, BSEE, BBA, and BS Mathematics across four room blocks (Block A, B, C, and D) offering rooms of varying capacities. The system manages student profiles, room allocation, attendance verification, entry and exit permissions, complaints, and notifications within a single normalized relational database.

The system operates in an educational hostile environment and supports four user roles: Students who reside in the hostel, the Warden (currently Ms. Nida Sultan Nehra) who is responsible for approvals and supervision, Security Personnel stationed at the gate who verify entries and exits, and System Administrators who maintain and configure the database. The hostel enforces a 10:00 PM curfew, after which all students must have marked their attendance and any approved exit must have been completed.

SHMS replaces fragmented manual processes paper attendance registers, physical exit slips, and disconnected room allocation spreadsheets with an integrated digital solution that provides real-time tracking, automated notifications, and auditable records.

1.2. PROBLEM STATEMENT:

The female hostel at Namal University currently relies on a largely manual management process, resulting in administrative delays, data redundancy, and security vulnerabilities. The following key problems have been identified:

- **Manual Attendance System:** Attendance is recorded manually on biometric machines by 10:00 PM. This approach allows proxy attendance, produces error-prone records, and requires wardens to manually cross-check against leave applications a slow and inconsistent process.
- **Paper-Based Exit Permissions:** Students use physical slips for entry and exit permissions, which creates security risks, makes tracking student movements difficult, and provides no real-time visibility to security personnel at the gate.
- **Lack of Real-Time Monitoring:** No digital system exists for monitoring room occupancy or tracking which students are currently inside or outside the hostel premises.
- **Disconnected Records:** Student information and room allocation records are maintained in separate spreadsheets, leading to data redundancy, inconsistent updates, and difficulty in generating consolidated reports.
- **No Automated Alerts:** There is no automated mechanism to notify wardens of absentees, late returns, or students who have not submitted required applications, increasing the administrative burden on hostel staff.

These challenges demand a centralized, normalized relational database that digitally handles attendance verification, entry and exit permissions, and room allocation with real-time accuracy and improved security.

1.3. PROJECT OBJECTIVES:

General Objective: To design and implement a normalized relational database that automates attendance verification, entry and exit approval workflows, and room allocation for the female hostel at Namal University, improving data accuracy, transparency, and administrative efficiency.

Specific Objectives:

- Design a Third Normal Form (3NF) compliant relational schema covering all hostel entities, including users, students, wardens, rooms, logs, geofence boundaries, and device registrations.
- Implement multi-factor attendance verification using GPS geofencing (Girls Hostel Boundary: latitude 33.6850, longitude 73.0485, radius 50 m) and selfie capture to eliminate proxy attendance by the end of the implementation phase.
- Enable digital submission and warden approval of exit and entry applications, reducing application processing time from hours to under 30 minutes.
- Maintain real-time room occupancy data through database triggers that automatically update Room. CurrentOccupancy whenever an allocation is created or deactivated.
- Provide role-based access control for Students, Wardens, Security Personnel, and System Administrators, ensuring each user accesses only the data their role permits.
- Implement stored procedures for all core CRUD operations to enforce consistent business-rule application regardless of which client web or mobile initiates the action.
- Generate automatic notification records for all status changes, including exit application approvals, rejections, and late return alerts.
- Maintain a complete, auditable System Change Log that records every administrative action along with the responsible administrator and timestamp.

GITHUB REPOSITORY LINK:

<https://github.com/Muhammad-Ahmad-99/Smart-Hostel-Management-System>

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY:

The following table identifies and defines the core entities of SHMS. Each entity represents a distinct real-world object or event that the system must track. Note: associative entities such as RoomAllocation and ComplaintAttachment are not listed here; they appear in the relational schema.

Sr. No	Entity Name	Description
01	User	An individual who has a registered account in the system. All roles Student, Warden, Security Personnel, and System Administrator are specializations of this entity sharing a common identity.
02	Student	A female resident of the hostel with an active room allocation. Specializes the User entity with registration number, degree program, and account status.
03	Warden	A hostel staff member responsible for reviewing exit applications, managing room allocations, and resolving complaints. Currently Ms. Nida Sultan Nehra (EMP-WRD-001).
04	System Administrator	A technical staff member responsible for database maintenance, user management, geofence configuration, and system auditing.
05	Security Personnel	A gate security staff member responsible for verifying student entries and exits against approved applications.
06	Room	A physical hostel room identified by a block and number (e.g., A01, B03), with a defined capacity and real-time occupancy count. Rooms exist in four blocks: Block A (2-person), Block B (4-person), Block C (6-person), and Block D (8-person).
07	Exit Application	A formal request submitted by a student to leave the hostel premises for a specified duration, subject to warden approval before departure.
08	Entry/Exit Log	A gate verification record capturing the actual exit and return timestamps for a student's approved trip, including a selfie for identity confirmation.
09	Attendance	A daily record of a student's verified presence within the hostel boundary, captured through GPS location check or selfie, with a 10:00 PM deadline.
10	Complaint	A maintenance or facility-related issue submitted by a student for warden resolution, categorized by type such as Maintenance, Internet, Water, or Roommate.
11	Notification	A system-generated alert or message sent to a user when a status change occurs, such as an exit application approval or a late return alert.
12	Geofence Boundary	A virtual geographic boundary used to validate a student's location during attendance verification. Key boundaries include Girls Hostel Boundary (radius 50 m) and Namal University Main Gate (radius 100 m).
13	Device Registration	A record of a student's registered mobile device, storing the device token used for push notification delivery.
14	System Change Log	An immutable audit record of every administrative action performed within the system, storing the action type, affected entity, and timestamp.

2.2. DATA DICTIONARY:

The following sub-sections list the attributes of each entity, including data type, constraints, and description. Foreign key attributes are excluded from this section in accordance with the template guidelines; they appear in the relational schema.

User:

Sr. No	Name	Data Type	Constraint	Description
01	UserID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique numeric identifier for every user account.
02	Name	VARCHAR(100)	NOT NULL	Full name of the user (e.g., Ayesha Khan, Nida Sultan Nehra).
03	Email	VARCHAR(100)	UNIQUE, NOT NULL	Namal University email used as login credential (e.g., bscs24f1@namal.edu.pk).
04	Password	VARCHAR(255)	NOT NULL	SHA-256 hashed password stored for secure authentication.
05	Phone	VARCHAR(20)	NOT NULL	11-digit Pakistani mobile number (e.g., 03001234567).
06	Role	ENUM	NOT NULL (Student, Warden, Admin, Security)	Determines which dashboard and access rights the user receives after login.
07	CreatedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of account creation; automatically set by the database.

Student:

Sr. No	Name	Data Type	Constraint	Description
01	StudentID (PK)	INT	NOT NULL	Shared with UserID; uniquely identifies the student record.
02	RegistrationNO	VARCHAR(50)	UNIQUE, NOT NULL	Namal University registration number (e.g., NUM-BSCS-2024-1).
03	DegreeYear	INT	CHECK (1 to 6)	Current academic year of the student, from 1 to 6.
04	DegreeProgram	VARCHAR(100)	—	Enrolled degree program: BSCS, BSEE, BBA, or BS Mathematics.
05	ProfilePicture	VARCHAR(255)	—	Server file path of the student's profile photo (e.g., profile_1.jpg).
06	AccountStatus	ENUM	DEFAULT 'Active' (Active, Inactive, Suspended)	Current hostel account status; Suspended blocks access without deletion.

Warden:

Sr. No	Name	Data Type	Constraint	Description
01	WardenID (PK)	INT	NOT NULL	Shared with UserID; uniquely identifies the warden record.
02	EmployeeID	VARCHAR(50)	UNIQUE, NOT NULL	Official employee code (e.g., EMP-WRD-001).
03	OfficePhone	VARCHAR(20)	—	Hostel office landline number (e.g., 0459-123456).
04	HireDate	DATE	—	Date the warden was appointed (e.g., 2024-01-15).
05	AccountStatus	ENUM	DEFAULT 'Active' (Active, Inactive)	Active means warden can log in and perform approvals.

Security Personnel:

Sr. No	Name	Data Type	Constraint	Description
01	SecurityID (PK)	INT	NOT NULL	Shared with UserID; uniquely identifies the security record.
02	EmployeeID	VARCHAR(50)	UNIQUE, NOT NULL	Official employee code (e.g., EMP-SEC-001, EMP-SEC-002, EMP-SEC-003).

System Administrator:

Sr. No	Name	Data Type	Constraint	Description
01	AdminID (PK)	INT	NOT NULL	Shared with UserID; uniquely identifies the admin record.
02	EmployeeID	VARCHAR(50)	UNIQUE, NOT NULL	Official employee code (e.g., EMP-ADM-001).
03	LastAuditDate	DATETIME	—	Timestamp of the most recent system audit performed by this administrator.

Room:

Sr. No	Name	Data Type	Constraint	Description
01	RoomID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique numeric identifier for the room.
02	RoomNumber	VARCHAR(20)	UNIQUE, NOT NULL	Display identifier using block-number format (e.g., A01, B03, C15, D08).
03	Capacity	INT	NOT NULL, CHECK > 0	Maximum students allowed: 2 for Block A, 4 for Block B, 6 for Block C, 8 for Block D.

Sr. No	Name	Data Type	Constraint	Description
04	CurrentOccupancy	INT	DEFAULT 0, CHECK: $0 \leq \text{value} \leq \text{Capacity}$	Maintained automatically by triggers on RoomAllocation insert and update.
05	RoomStatus	ENUM	DEFAULT 'Available' (Available, Occupied, Maintenance, Cleaning)	Reflects current usability of the room.

Exit Application:

Sr. No	Name	Data Type	Constraint	Description
01	ApplicationID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each exit request.
02	Destination	TEXT	—	Student-stated destination (e.g., Mianwali City, home address).
03	IsOnLeave	BOOLEAN	DEFAULT FALSE	TRUE for multi-day leave; FALSE for a same-day exit.
04	Reason	TEXT	—	Brief justification provided by the student for the exit.
05	DepartureTime	DATETIME	NOT NULL	Planned date and time of departure from hostel.
06	ExpectedReturnTime	DATETIME	NOT NULL	Planned date and time of return; used to trigger late-return alert.
07	RestrictedTimeFlag	BOOLEAN	DEFAULT FALSE	Automatically set TRUE if departure or return time falls outside permitted hours.
08	SubmittedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when the student submitted the application.
09	ApprovalStatus	ENUM	DEFAULT 'Pending' (Pending, Approved, Rejected, Cancelled)	Changes trigger an automatic notification to the student.
10	WardenRemarks	TEXT	—	Comments entered by the warden during review (e.g., 'Approved - Return before 5 PM').

Entry/Exit Log:

Sr. No	Name	Data Type	Constraint	Description
01	LogID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each gate verification event.
02	ExitTime	DATETIME	—	Actual time the student was verified and allowed to exit the gate.
03	ActualReturnTime	DATETIME	—	Actual time the student was verified upon return to the hostel.
04	Selfie	VARCHAR(255)	—	File path of the selfie captured at the gate for identity verification.
05	LateReturnAlert	BOOLEAN	DEFAULT FALSE	Set TRUE automatically by trigger when ActualReturnTime > ExpectedReturnTime.

Attendance:

Sr. No	Name	Data Type	Constraint	Description
01	AttendanceID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each attendance record.
02	Date	DATE	NOT NULL	Calendar date of the attendance record (e.g., 2026-06-12).
03	Time	TIME	—	Time the attendance was marked (e.g., 08:15:00).
04	ApplicationStatus	ENUM	NOT NULL (Present, Absent, Late, Excused)	Outcome of the daily verification check.
05	Selfie	VARCHAR(255)	—	File path of the selfie captured during verification (e.g., att_1_0612.jpg).
06	VerificationMethod	ENUM	NOT NULL (GPS, Face, Manual)	Method used: GPS for geofencing check, Face for selfie match, Manual for warden override.
07	GPSLatitude	DECIMAL(10,8)	—	Latitude recorded at verification (e.g., 33.68440000 for the hostel boundary).
08	GPSLongitude	DECIMAL(11,8)	—	Longitude recorded at verification (e.g., 73.04790000).
09	SuspiciousFlag	BOOLEAN	DEFAULT FALSE	Set TRUE if GPS coordinates fall outside geofence boundary or verification pattern is anomalous.

Sr. No	Name	Data Type	Constraint	Description
10	UNIQUE (StudentID, Date)	—	Composite constraint	Prevents more than one attendance record per student per day.

Complaint:

Sr. No	Name	Data Type	Constraint	Description
01	ComplaintID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each complaint.
02	Title	VARCHAR(200)	NOT NULL	Short descriptive title (e.g., 'AC not working', 'WiFi issue').
03	Description	TEXT	—	Full detail of the complaint (e.g., 'Room AC is not cooling properly since 3 days').
04	ComplaintType	ENUM	NOT NULL (Maintenance, Roommate, Room, Electricity, Water, Internet, Mess, Security, Other)	Category used for routing and reporting.
05	Status	ENUM	DEFAULT 'Open' (Open, InProgress, Resolved, Rejected)	Current resolution status of the complaint.
06	SubmittedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of submission.
07	ResolvedAt	DATETIME	—	Timestamp when complaint was marked Resolved (e.g., 2026-05-22 14:00:00).
08	WardenRemarks	TEXT	—	Comments from warden on resolution (e.g., 'Technician repaired AC').

Notification:

Sr. No	Name	Data Type	Constraint	Description
01	NotificationID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each notification.
02	Type	ENUM	NOT NULL (Alert, Reminder, Info, Warning, Emergency)	Category of notification determining display priority.
03	Message	TEXT	NOT NULL	Full notification text (e.g., 'Your exit application #3 has been Approved').
04	SentAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp the notification was generated.
05	IsRead	BOOLEAN	DEFAULT FALSE	Set TRUE when the recipient opens the notification in the app.

Geofence Boundary:

Sr. No	Name	Data Type	Constraint	Description
01	GeofenceID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each boundary.
02	BoundaryName	VARCHAR(100)	UNIQUE, NOT NULL	Descriptive name (e.g., 'Girls Hostel Boundary', 'Namal University Main Gate').
03	CenterLatitude	DECIMAL(10,8)	NOT NULL	Latitude of boundary centre (e.g., 33.68500000 for Girls Hostel Boundary).
04	CenterLongitude	DECIMAL(11,8)	NOT NULL	Longitude of boundary centre (e.g., 73.04850000 for Girls Hostel Boundary).
05	RadiusMeters	DECIMAL(10,2)	—	Radius defining the circular boundary (e.g., 50.00 m for hostel, 100.00 m for main gate).
06	CreatedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of boundary creation.

Device Registration:

Sr. No	Name	Data Type	Constraint	Description
01	DeviceID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each registered device.
02	DeviceToken	VARCHAR(255)	UNIQUE, NOT NULL	Firebase or APNs push notification token for the device.
03	DeviceType	ENUM	NOT NULL (Android, iOS, Web, Tablet)	Operating platform of the registered device.
04	LastActiveAt	DATETIME	—	Timestamp of the device's most recent activity in the system.
05	RegisteredAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp when the device was first registered.

System Change Log:

Sr. No	Name	Data Type	Constraint	Description
01	logID (PK)	INT	AUTO_INCREMENT, NOT NULL	Unique identifier for each audit record.
02	Action	VARCHAR(100)	NOT NULL	Type of operation performed: INSERT, UPDATE, or DELETE.
03	EntityName	VARCHAR(100)	NOT NULL	Name of the database table affected (e.g., GeofenceBoundary, RoomAllocation).
04	RecordID	INT	—	Primary key value of the specific record affected.
05	TimeDate	DATETIME	DEFAULT CURRENT_TIMESTAMP	Exact timestamp of the administrative action.
06	Description	TEXT	—	Narrative description of the change (e.g., 'Geofence radius updated from 80 to 100 meters').

2.3. BUSINESS RULES:

The following business rules define the conditions, constraints, and logic that govern how data is created, managed, and related within SHMS:

- Each User must have a unique Email address and a Role of exactly one of: Student, Warden, Admin, or Security.
- Every Student, Warden, System Administrator, and Security Personnel record must correspond to exactly one User record, sharing the same primary key value.
- Deleting a User record automatically removes the corresponding role-specific record through cascading delete.
- A student may have at most one active room allocation at any point in time, enforced through a unique constraint on (StudentID, Status='Active') in RoomAllocation.
- A room's CurrentOccupancy must never exceed its Capacity, enforced by a CHECK constraint; the system must reject any allocation that would violate this rule.
- When a new RoomAllocation record is inserted, the corresponding Room's CurrentOccupancy is automatically incremented by one via a database trigger.
- When an active RoomAllocation's Status changes to 'Inactive', the corresponding Room's CurrentOccupancy is automatically decremented by one via a database trigger.
- Every ExitApplication must be associated with exactly one student. A warden is assigned only upon review; the WardenID is nullable until that point.
- An ExitApplication's ApprovalStatus must be exactly one of: Pending, Approved, Rejected, or Cancelled. Status can only move forward, not backward.
- Whenever an ExitApplication's ApprovalStatus changes, a Notification record is automatically inserted for the student via a database trigger.

- An EntryExitLog entry must always reference a student. It may optionally reference an ExitApplication; if the referenced application is deleted, ApplicationID is set to NULL.
- If a student's ActualReturnTime exceeds the ExpectedReturnTime on the linked ExitApplication, the LateReturnAlert flag in EntryExitLog is automatically set to TRUE by a trigger.
- Each student may have at most one Attendance record per calendar date, enforced through a unique constraint on (StudentID, Date).
- Every Attendance record must specify a VerificationMethod of GPS, Face, or Manual. GPS and Face are captured through the mobile app; Manual is used for warden-override cases.
- The SuspiciousFlag in Attendance is set TRUE if GPS coordinates fall outside the Girls Hostel Boundary (radius 50 m, centre 33.6850 / 73.0485) or if an anomalous verification pattern is detected.
- A Complaint must belong to exactly one student. It may have zero or more ComplaintAttachment records, which are deleted automatically if the complaint is deleted.
- Deleting a User removes all associated Notification records. A Notification may be linked to exactly one user.
- Every SystemChangeLog entry must reference the System Administrator who performed the action; the AdminID is non-nullable.

2.4. ENTITY RELATIONSHIP DIAGRAM:

The Entity-Relationship Diagram for SHMS was developed using diagrams.net (draw.io), based on requirements gathered from hostel administration interviews, review of manual records, and team analysis. The diagram uses crow's foot notation to represent cardinalities and captures the ISA (specialization) relationship between User and the four role-specific entities, as well as all transactional and associative relationships.

Key relationships and their cardinalities are:

- User to Student / Warden / System Administrator / Security Personnel (1:1, ISA): each user is specialized into exactly one role entity sharing the UserID as a foreign primary key.
- Student to Attendance (1:M): one student has many attendance records over time; each record belongs to exactly one student, with at most one per calendar date.
- Student to RoomAllocation (1:M): a student may have a history of allocation records but only one may be Active at any time; each allocation belongs to one student.
- Room to RoomAllocation (1:M): one room can appear in many allocation records over time; each allocation record refers to exactly one room.
- Student to ExitApplication (1:M): one student may submit many exit applications; each application belongs to one student.
- Warden to ExitApplication (1:M, reviews): one warden may review many applications; each application is reviewed by at most one warden (nullable).
- ExitApplication to EntryExitLog (1:M): one approved application may produce multiple gate log entries for exit and return events.
- Student to Complaint (1:M): one student may file many complaints; each complaint belongs to one student.

- Complaint to ComplaintAttachment (1:M): one complaint may have many file attachments; each attachment belongs to one complaint.
- User to Notification (1:M): one user may receive many notifications; each notification is addressed to one user.
- GeofenceBoundary to GeofencePolygonPoints (1:M): one boundary may have many polygon vertex points defining its shape.
- Student to DeviceRegistration (1:M): one student may register many devices for push notifications.
- System Administrator to SystemChangeLog (1:M): one administrator's actions produce many audit log entries.

[Figure 1: Entity Relationship Diagram of the Smart Hostel Management System \(SHMS\)](#)

[Figure 2: Relational Schema Diagram of the Smart Hostel Management System \(SHMS\)](#)

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA:

The conceptual ERD was converted into the following relational schema using the standard ERD-to-relational mapping rules: each entity became a table, single-valued attributes became columns, primary keys were preserved, one-to-many relationships were implemented by placing a foreign key on the 'many' side, and associative entities (RoomAllocation, ComplaintAttachment) were retained as full relations carrying their own descriptive attributes. Primary keys are marked PK; foreign keys are marked FK with the referenced table.

- User(UserID PK, Name, Email, Password, Phone, Role, CreatedAt)
- Student(StudentID PK / FK → User.UserID, RegistrationNO, DegreeYear, DegreeProgram, ProfilePicture, AccountStatus)
- Warden(WardenID PK / FK → User.UserID, EmployeeID, OfficePhone, HireDate, AccountStatus)
- SystemAdministrator(AdminID PK / FK → User.UserID, EmployeeID, LastAuditDate)
- SecurityPersonnel(SecurityID PK / FK → User.UserID, EmployeeID)
- Room(RoomID PK, RoomNumber, Capacity, CurrentOccupancy, RoomStatus)
- RoomAllocation(AllocationID PK, StudentID FK → Student.StudentID, RoomID FK → Room.RoomID, AllocationType, AllocationDate, Status)
- ExitApplication(ApplicationID PK, StudentID FK → Student.StudentID, Destination, IsOnLeave, Reason, DepartureTime, ExpectedReturnTime, RestrictedTimeFlag, SubmittedAt, ApprovalStatus, WardenID FK → Warden.WardenID [nullable], WardenRemarks)
- EntryExitLog(LogID PK, ApplicationID FK → ExitApplication.ApplicationID [nullable, SET NULL on delete], StudentID FK → Student.StudentID, ExitTime, ActualReturnTime, Selfie, LateReturnAlert)

- Attendance(AttendanceID PK, StudentID FK → Student.StudentID, Date, Time, ApplicationStatus, Selfie, VerificationMethod, GPSLatitude, GPSLongitude, SuspiciousFlag, UNIQUE(StudentID, Date))
- Complaint(ComplaintID PK, StudentID FK → Student.StudentID, Title, Description, ComplaintType, Status, SubmittedAt, ResolvedAt, WardenRemarks)
- ComplaintAttachment(AttachmentID PK, ComplaintID FK → Complaint.ComplaintID, FilePath, FileName, UploadedAt)
- Notification(NotificationID PK, UserID FK → User.UserID [nullable, CASCADE on delete], Type, Message, SentAt, IsRead)
- GeofenceBoundary(GeofenceID PK, BoundaryName, CenterLatitude, CenterLongitude, RadiusMeters, CreatedAt)
- GeofencePolygonPoints(PointID PK, GeofenceID FK → GeofenceBoundary.GeofenceID, Latitude, Longitude, PointOrder)
- DeviceRegistration(DeviceID PK, StudentID FK → Student.StudentID, DeviceToken, DeviceType, LastActiveAt, RegisteredAt)
- SystemChangeLog(logID PK, AdminID FK → SystemAdministrator.AdminID, Action, EntityName, RecordID, TimeDate, Description)

No additional bridge tables are required. All many-to-many relationships in the conceptual model resolve into associative entities (RoomAllocation, ComplaintAttachment) that carry their own primary keys and descriptive attributes.

3.2. FUNCTIONAL DEPENDENCIES:

For each table in the relational schema, the functional dependencies are listed below using the format $A \rightarrow B$, meaning attribute A functionally determines attribute B. Concrete examples using real project data are provided to illustrate each dependency.

Table	Functional Dependency	Concrete Example
User	UserID → Name, Email, Phone, Role, CreatedAt	UserID 1 → 'Ayesha Khan', 'bscs24f1@namal.edu.pk', '03001234567', 'Student'
Student	StudentID → RegistrationNO, DegreeYear, DegreeProgram, AccountStatus	StudentID 1 → 'NUM-BSCS-2024-1', 1, 'BSCS', 'Active'
Warden	WardenID → EmployeeID, OfficePhone, HireDate, AccountStatus	WardenID 51 → 'EMP-WRD-001', '0459-123456', '2024-01-15', 'Active'
Room	RoomID → RoomNumber, Capacity, CurrentOccupancy, RoomStatus	RoomID 1 → 'A01', 2, 2, 'Occupied'
RoomAllocation	AllocationID → StudentID, RoomID, AllocationType, AllocationDate, Status	AllocationID 1 → StudentID 1, RoomID 1, 'Permanent', '2024-08-20', 'Active'
ExitApplication	ApplicationID → StudentID, Destination, DepartureTime, ExpectedReturnTime, ApprovalStatus	ApplicationID 1 → StudentID 1, 'Mianwali City', '2026-06-01 14:00', '2026-06-01 18:00', 'Approved'

Table	Functional Dependency	Concrete Example
EntryExitLog	$\text{LogID} \rightarrow \text{ApplicationID}, \text{StudentID}, \text{ExitTime}, \text{ActualReturnTime}, \text{LateReturnAlert}$	$\text{LogID } 1 \rightarrow \text{AppID } 1, \text{StudentID } 1, '2026-06-01 14:05', '2026-06-01 17:50', \text{FALSE}$
Attendance	$\text{AttendanceID} \rightarrow \text{StudentID}, \text{Date}, \text{Time}, \text{ApplicationStatus}, \text{VerificationMethod}, \text{GPSLatitude}, \text{GPSLongitude}$	$\text{AttendanceID } 1 \rightarrow \text{StudentID } 1, '2026-06-12', '08:15:00', 'Present', 'GPS', 33.6844, 73.0479$
Complaint	$\text{ComplaintID} \rightarrow \text{StudentID}, \text{Title}, \text{ComplaintType}, \text{Status}, \text{SubmittedAt}$	$\text{ComplaintID } 1 \rightarrow \text{StudentID } 1, 'AC not working', 'Maintenance', 'Resolved', '2026-05-20'$
GeofenceBoundary	$\text{GeofenceID} \rightarrow \text{BoundaryName}, \text{CenterLatitude}, \text{CenterLongitude}, \text{RadiusMeters}$	$\text{GeofenceID } 2 \rightarrow 'Girls Hostel Boundary', 33.6850, 73.0485, 50.00$
DeviceRegistration	$\text{DeviceID} \rightarrow \text{StudentID}, \text{DeviceToken}, \text{DeviceType}, \text{LastActiveAt}$	$\text{DeviceID } 1 \rightarrow \text{StudentID } 1, '<\text{token}>', 'Android', '2026-06-13 10:00'$
SystemChangeLog	$\text{logID} \rightarrow \text{AdminID}, \text{Action}, \text{EntityName}, \text{RecordID}, \text{TimeDate}, \text{Description}$	$\text{logID } 1 \rightarrow \text{AdminID } 55, 'UPDATE', 'GeofenceBoundary', 1, '2026-01-15', 'Radius updated 80 \rightarrow 100 \text{ m}'$

Additional dependency worth noting: within Attendance, the combination (StudentID, Date) is a candidate key because of the unique constraint, meaning $\text{StudentID} + \text{Date} \rightarrow \text{Time}, \text{ApplicationStatus}, \text{Selfie}, \text{VerificationMethod}, \text{GPSLatitude}, \text{GPSLongitude}, \text{SuspiciousFlag}$. However, since a surrogate primary key (AttendanceID) exists, no composite primary key is required.

3.3. NORMALIZATION:

The relational schema of the Smart Hostel Management System (SHMS) was evaluated using the functional dependencies identified, to ensure compliance with normalization principles. The normalization process was carried out up to Third Normal Form (3NF) to eliminate redundancy and maintain data integrity.

3.3.1 First Normal Form (1NF)

A relation is considered to be in First Normal Form (1NF) when all attributes contain atomic values and no repeating groups exist.

All relations in the SHMS schema satisfy the requirements of 1NF. The following design decisions ensure compliance:

- Multiple complaint attachments are not stored within a single attribute. Instead, each attachment is represented as a separate record in the ComplaintAttachment relation.
- Each student record contains a single PhoneNumber, while each warden record contains a single OfficePhone value.
- Attendance and entry/exit records store only one Selfie image reference per record.
- All attributes across the sixteen relations contain indivisible, single-valued data elements, and no repeating groups are present.

Therefore, all SHMS relations are in First Normal Form.

3.3.2 Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if it is already in 1NF and every non-key attribute is fully functionally dependent on the entire primary key.

Since all SHMS relations use single-attribute primary keys, partial dependencies cannot occur. Consequently, every non-key attribute depends completely on its respective primary key.

Relation	Primary Key	2NF Status
User	UserID	✓
Student	StudentID	✓
Warden	WardenID	✓
SystemAdministrator	AdminID	✓
SecurityPersonnel	SecurityID	✓
Room	RoomID	✓
RoomAllocation	AllocationID	✓
ExitApplication	ApplicationID	✓
Attendance	AttendanceID	✓
EntryExitLog	LogID	✓
Complaint	ComplaintID	✓
ComplaintAttachment	AttachmentID	✓
Notification	NotificationID	✓
DeviceRegistration	DeviceID	✓
GeofenceBoundary	GeofenceID	✓
SystemChangeLog	LogID	✓

Hence, all relations satisfy Second Normal Form.

3.3.3 Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and contains no transitive dependencies, meaning that no non-key attribute depends on another non-key attribute.

The SHMS schema satisfies 3NF through the following design considerations:

- Role-specific attributes are separated into the subtype relations Student, Warden, SystemAdministrator, and SecurityPersonnel instead of being stored in the User relation. This removes transitive dependencies associated with the user role.
- Complaint attachments are stored in the ComplaintAttachment relation rather than within the Complaint relation, ensuring that all attributes depend directly on their respective primary keys.
- In ExitApplication, the attribute WardenRemarks depends directly on ApplicationID, since different applications reviewed by the same warden may contain different remarks.
- In Attendance, the attribute GPSLocation depends directly on AttendanceID and not on any other non-key attribute.
- In RoomAllocation, the attribute Status depends solely on AllocationID.
- In EntryExitLog, the attribute LateReturnAlert is determined by LogID rather than by StudentID or ApplicationID.
- In Notification, the attribute IsRead depends directly on NotificationID.
- In DeviceRegistration, DeviceType is determined by DeviceID.
- In SystemChangeLog, Description depends directly on LogID rather than on AdminID.

As a result, no transitive dependencies exist within any relation, and all relations satisfy Third Normal Form.

3.3.4 Normalization Summary

Relation	1NF	2NF	3NF
User	✓	✓	✓
Student	✓	✓	✓
Warden	✓	✓	✓
SystemAdministrator	✓	✓	✓
SecurityPersonnel	✓	✓	✓
Room	✓	✓	✓
RoomAllocation	✓	✓	✓
ExitApplication	✓	✓	✓
Attendance	✓	✓	✓
EntryExitLog	✓	✓	✓
Complaint	✓	✓	✓

Relation	1NF	2NF	3NF
ComplaintAttachment	✓	✓	✓
Notification	✓	✓	✓
DeviceRegistration	✓	✓	✓
GeofenceBoundary	✓	✓	✓
SystemChangeLog	✓	✓	✓

3.3.5 Conclusion

The normalization analysis confirms that the SHMS database schema contains no repeating groups, partial dependencies, or transitive dependencies. Consequently, all sixteen relations comply with the requirements of Third Normal Form (3NF). The resulting schema provides reduced redundancy, improved consistency, and enhanced data integrity for the Smart Hostel Management System.

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/Framework:

Database Management System: MySQL 8.0 was selected as the RDBMS for SHMS. MySQL was chosen because of its support for relational database features such as foreign keys, views, stored procedures, triggers, and events. The database serves as the central repository for all hostel operations including attendance management, exit applications, room allocation, complaint handling, notifications, and administrative settings. Business rules are enforced through database objects to ensure data consistency and integrity throughout the system.

Backend Framework: Python 3.10 with Flask 3.0.3 was selected for the application layer. Flask handles routing, authentication, role-based access control, session management, and communication with the MySQL database through the PyMySQL library. Password security is implemented using Werkzeug password hashing mechanisms.

Frontend: HTML5, CSS3, JavaScript, and Bootstrap were used to develop a responsive web interface. The system provides separate dashboards for Students, Wardens, Security Personnel, and Administrators.

Additional Technologies: Browser APIs are used for camera access and GPS location verification. Attendance marking requires selfie capture and geofence validation through the browser. The application is accessible through modern web browsers such as Chrome, Firefox, and Edge.

4.2. DATABASE CONNECTIVITY:

The Smart Hostel Management System follows a three-tier architecture consisting of the user interface, Flask application server, and MySQL database.

The web interface communicates with the Flask application through HTTP requests. Flask processes user requests, performs validation and authentication, and interacts with the MySQL database using the PyMySQL library. Database connection parameters such as host, database name, username, and password are maintained within the application configuration.

Connection Flow:

GUI Action → HTTP Request → Flask Application (app.py) → PyMySQL Database Connection → MySQL Database → Result Set → Flask Response → GUI Output

The database acts as the single source of truth for all hostel operations. Stored procedures, triggers, views, and events ensure that business rules are enforced consistently regardless of the user role performing an operation.

4.3. STORED PROCEDURES AND FUNCTIONS:

The SHMS database utilizes advanced database objects to support automation, data integrity, and efficient data management. All major operations related to attendance, room allocation, complaints, exit applications, notifications, and administrative management are handled through stored procedures and supporting database objects.

Object Type	Count
Tables	15
Views	6
Stored Procedures	18
Functions	3
Triggers	2
Events	3

Stored Procedures:

Stored procedures are used to perform operations such as student management, attendance recording, room allocation, complaint submission and resolution, exit application processing, report generation, and administrative tasks. These procedures centralize business logic and reduce redundancy in application code.

Stored Functions:

Functions are used to perform reusable calculations and return single values. They support reporting and data retrieval operations by encapsulating commonly used calculations within the database.

Triggers:

Triggers automate database actions whenever specific events occur. Examples include automatic room occupancy updates and notification generation when system records are modified.

Views:

Views provide simplified and secure access to frequently used information by combining data from multiple tables. They support reporting, monitoring, and dashboard functionality.

Events:

MySQL events automate scheduled operations such as attendance processing, exit status updates, and other time-based system activities.

These database objects improve maintainability, consistency, security, and automation within the Smart Hostel Management System.

4.4. INTERFACES:

The following sub-sections describe the key screens of the SHMS GUI application. Screenshots should be inserted at the indicated positions once the frontend implementation is complete.

4.4.1 Login Screen

Presents Email and Password fields with a 'Login' button. On successful authentication, the Flask API checks User.Role and redirects the user to their role-specific dashboard. Invalid credentials display an error message without revealing whether the email or password is incorrect.

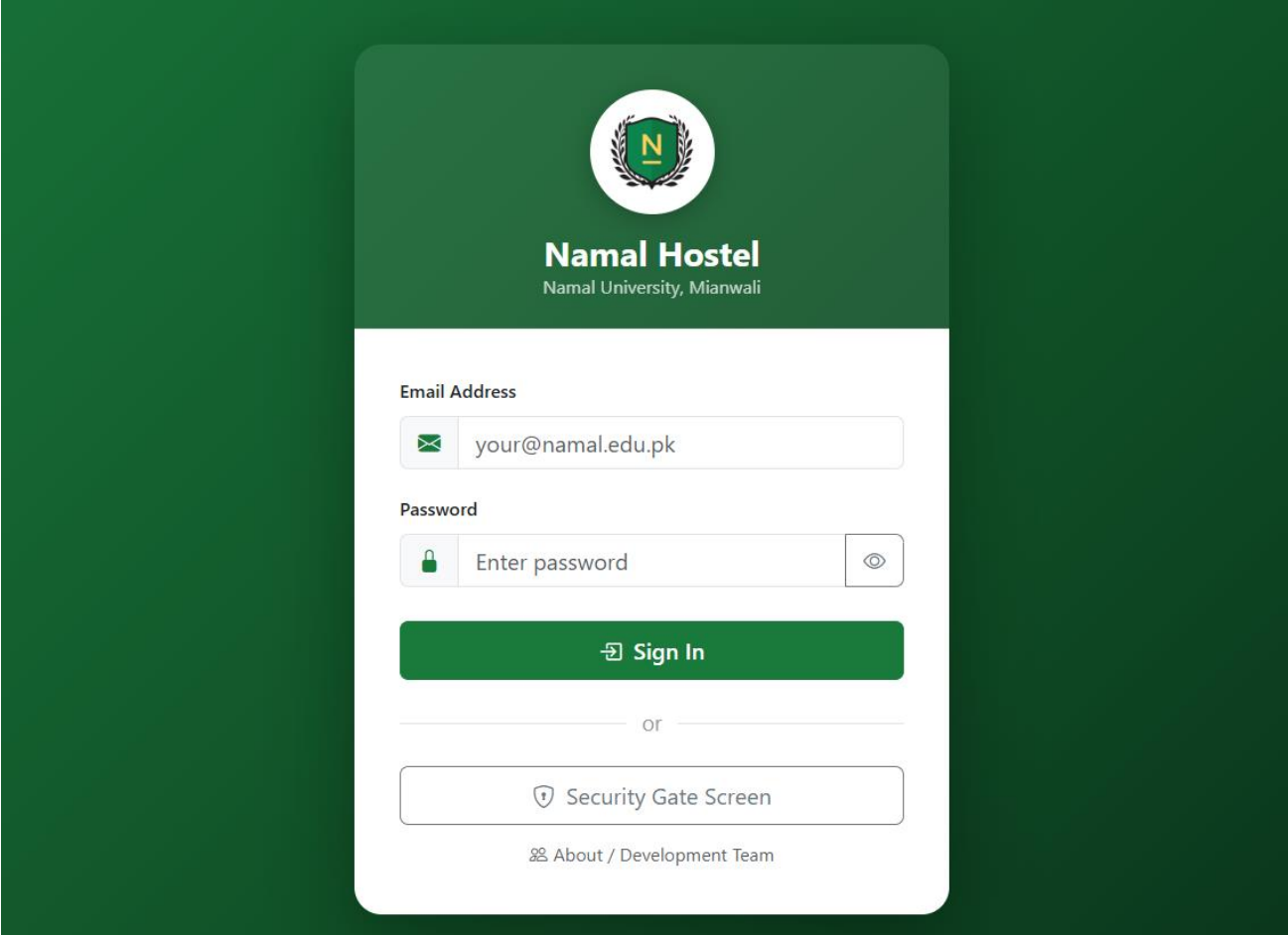


Figure 2: Login screen entry point for all user roles

4.4.2 Student Dashboard

Displays the student's name, registration number, profile picture, current room number, and today's attendance status. A notification bell shows unread alerts. Navigation links lead to: Submit Exit Application, View Attendance History, File a Complaint. The Mark Attendance button triggers the GPS/selfie verification workflow.

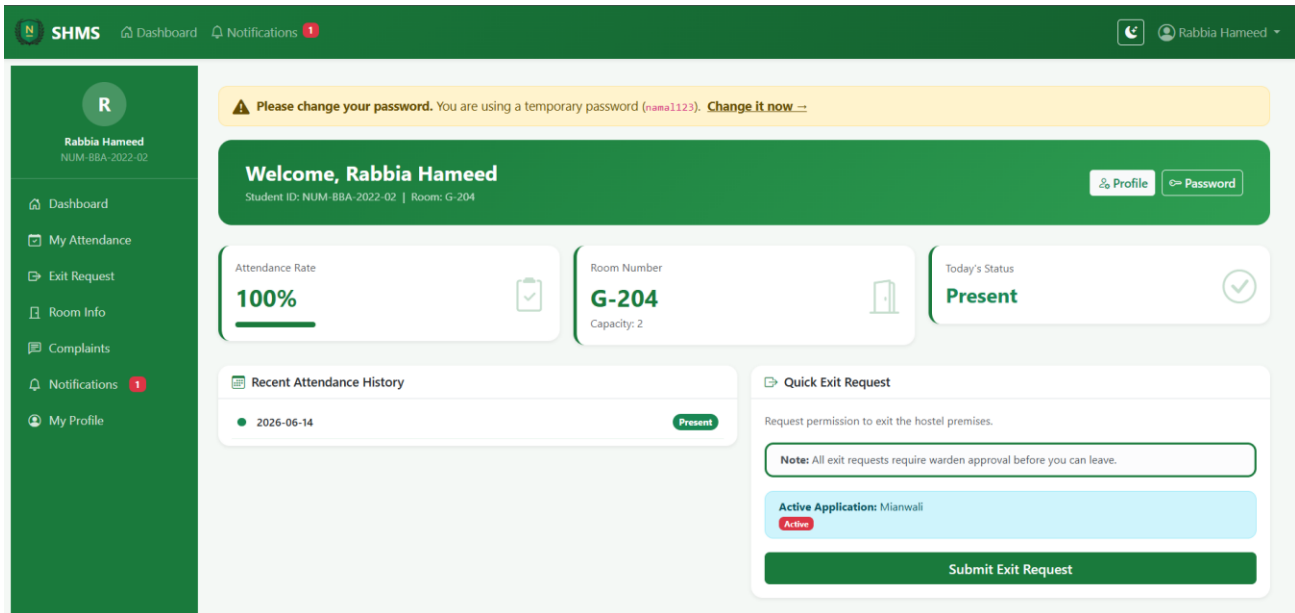


Figure 3: Student dashboard showing profile, room, attendance, and notifications

4.4.3 Warden Dashboard

Displays a list of pending exit applications fetched via GetPendingExits, showing StudentName, RegistrationNO, Destination, DepartureTime. The warden selects an application, enters optional remarks, and clicks Approve or Reject which calls ReviewExitApplication and automatically triggers a notification to the student. A second tab shows room occupancy across all four blocks with color-coded status indicators.

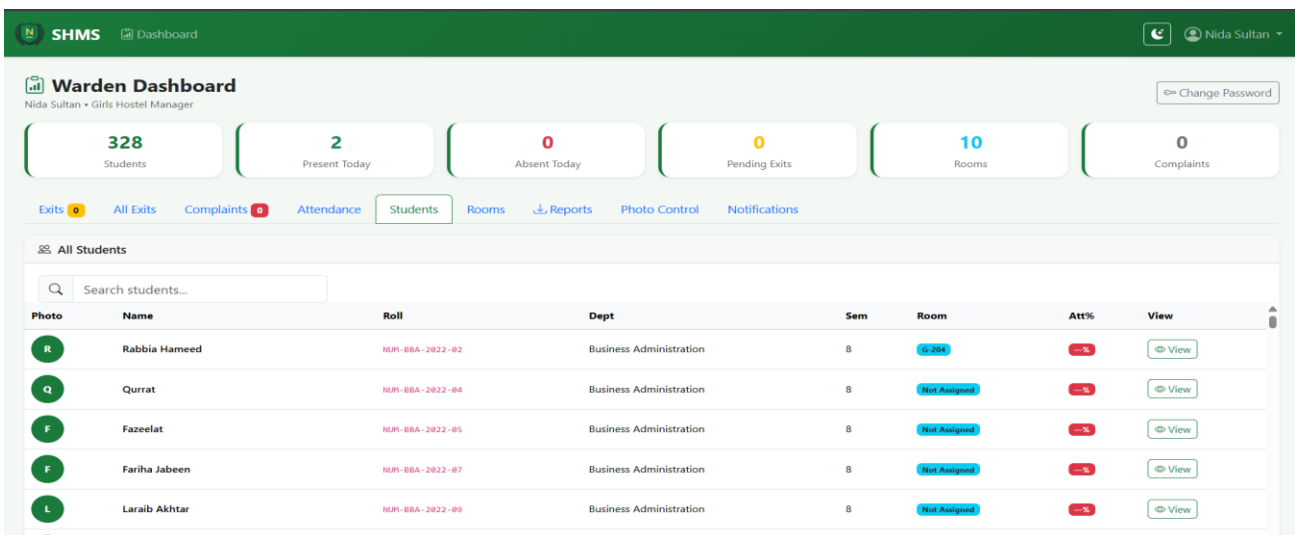


Figure 4: Warden panel pending exit applications and room occupancy overview

4.4.4 Attendance

For Students: a 'Mark Attendance' button captures the current GPS location and optionally a selfie, then calls MarkAttendance. The screen confirms success or displays an error if the student is outside the geofence boundary. For Wardens: a daily attendance summary table shows each student's status (Present, Absent, Late, Excused) with a filter for SuspiciousFlag records requiring manual review.

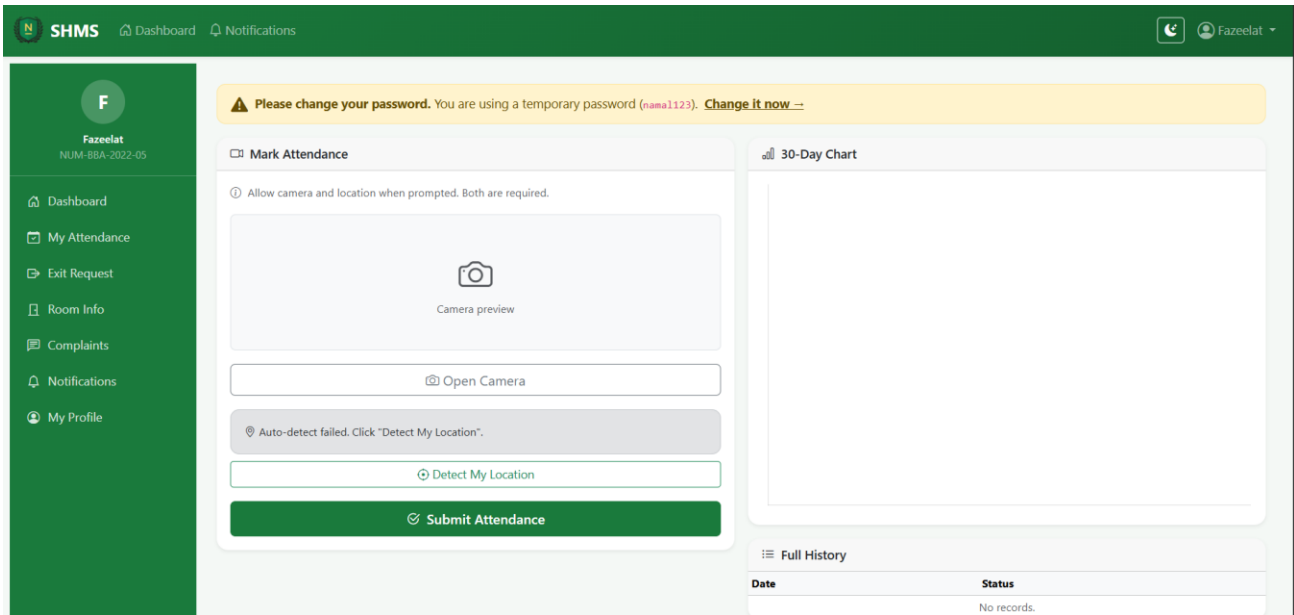


Figure 5: Attendance verification screen (student view) and daily summary (warden view)

4.4.5 Room Allocation

Displays a grid view of all 80 rooms across four blocks, with RoomNumber, Capacity, CurrentOccupancy, and RoomStatus shown for each. Rooms with Available status are highlighted. The warden or admin selects a student (searched by name or registration number) and an available room, then confirms allocation via allocateRoom. The trigger automatically updates CurrentOccupancy and RoomStatus.

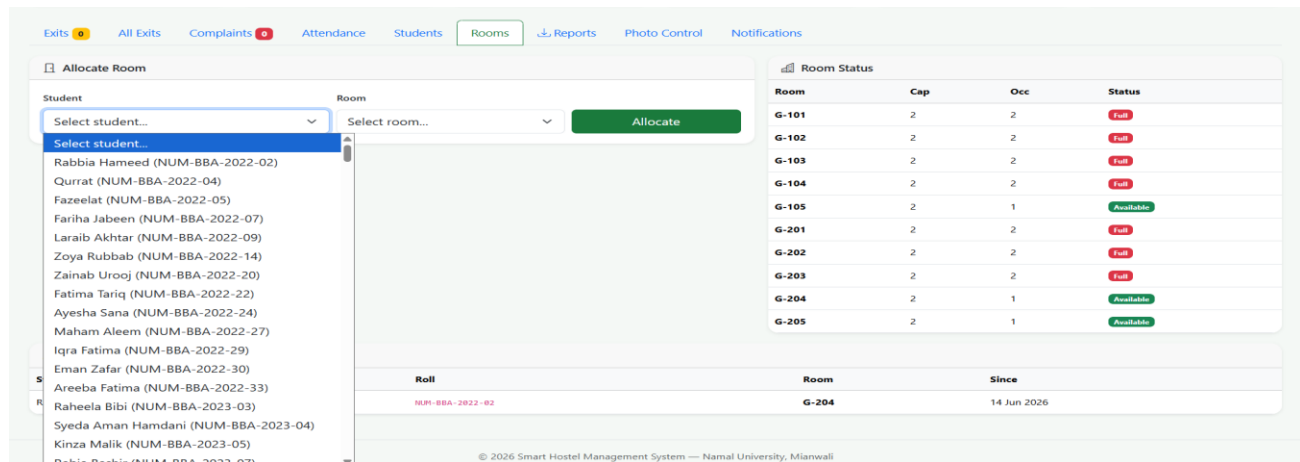


Figure 6: Room allocation screen showing block-wise occupancy grid

4.4.6 Exit Approval

Lists exit applications filterable by ApprovalStatus. Each row shows StudentName, RegistrationNO, Destination, DepartureTime, ExpectedReturnTime, and current ApprovalStatus. The warden clicks a row to expand it, enters remarks (e.g., 'Approved - Return before 5 PM'), and selects Approve, Reject, or Cancel. The action calls ReviewExitApplication, and the trigger auto-generates the student notification.

The screenshot shows the 'SHMS Warden Dashboard' for Nida Sultan, Girls Hostel Manager. The dashboard includes a top navigation bar with 'SHMS' and 'Dashboard' links, and a user profile 'Nida Sultan'. Below the navigation bar, there are six summary cards: '328 Students', '2 Present Today', '0 Absent Today', '0 Pending Exits', '10 Rooms', and '0 Complaints'. A 'Change Password' button is located next to the user profile. Below the summary cards, there are tabs for 'Exits', 'All Exits', 'Complaints', 'Attendance', 'Students', 'Rooms', 'Reports', 'Photo Control', and 'Notifications'. The 'All Exits' tab is selected, showing a table of exit applications. The table has columns for Student, Roll, Destination, Type, Departure, and Status. Two rows are visible: one for 'Qurrat' (Roll: NUM-BBA-2022-04, Destination: ht, Type: Short Exit, Departure: 14 Jun 16:57, Status: Active) and one for 'Rabbia Hameed' (Roll: NUM-BBA-2022-02, Destination: Mianwali, Type: Short Exit, Departure: 14 Jun 16:38, Status: Active). A copyright notice at the bottom reads '© 2026 Smart Hostel Management System — Namal University, Mianwali'.

Student	Roll	Destination	Type	Departure	Status
Qurrat	NUM-BBA-2022-04	ht	Short Exit	14 Jun 16:57	Active
Rabbia Hameed	NUM-BBA-2022-02	Mianwali	Short Exit	14 Jun 16:38	Active

Figure 7: Exit approval screen with application details and warden action controls

4.4.7 Security Gate Verification Screen

Displays the ApprovedExitView a read-only list of approved applications with StudentName, RollNumber, and DepartureTime. Security personnel search by name or roll number, confirm the student's identity, and log the actual exit via VerifyExit or the return via VerifyReturn. A late return alert is displayed if the check_late_return trigger fires.

The screenshot shows the 'Security Gate' screen for Girls Hostel. The top navigation bar includes 'Security Gate', 'Girls Hostel', and a language selector 'اردو'. There are also buttons for 'Active Exits' (with a red notification badge), 'History', and a search icon. Below the navigation bar, there is a timestamp '14-Jun-2026, 4:49:18 pm' and a search bar with the placeholder 'Search name / roll...'. The main content area displays a card for 'Rabbia Hameed' (Roll: NUM-BBA-2022-02, Business Administration). The card shows a 'Short Exit' button and a 'Ready' button. Below these buttons, it says 'Departure: 14 Jun 16:38'. At the bottom of the card, there is a 'Confirm Exit' button.

Figure 8: Security gate screen for exit and return verification

REFERENCES

- [1] Pallets Projects, "Flask Documentation (3.x)," 2026. [Online]. Available: <https://flask.palletsprojects.com/>
- [2] M. Ahmad, A. U. Khan, and M. Rashid, "Smart Hostel Management System GitHub Repository," 2026. [Online]. Available: <https://github.com/Muhammad-Ahmad-99/Smart-Hostel-Management-System>
- [3] Department of Software Engineering, Namal University, "Software Requirements Specification Smart Hostel Management System, 3rd Semester SE Project," 2025.

APPENDIX: AI USAGE AND PROMPTS

This section documents the use of AI tools during the development of this document and the associated database design. All AI-generated content was reviewed, verified against the actual database schema (SHMS_DataBase.sql) and sample data (SHMS_Sample_data.sql), and edited by the team before inclusion.

Tools Used: ChatGPT-4, DeepSeek, Claude, GitHub Copilot.

Sr. No	Prompt Used	Purpose
01	How to implement a fraud-proof attendance system using biometric devices and alternatives to this?	Researching GPS geofencing and selfie verification approaches for the Attendance entity design.
02	MySQL database schema for hostel management with room allocation and attendance tracking.	Initial schema structure and data type selection for core entities.
03	What should be included in the methodology and introduction section for a database project report?	Guidance for writing Sections 1.1 and 1.2.
04	How to design an approval workflow for entry/exit applications in a women's hostel?	Designing the ExitApplication entity and warden approval workflow.
05	What are the best practices for role-based access control in student housing systems?	Designing the User ISA specialization structure.
06	Anti-cheating mechanisms for student attendance tracking geofencing and randomized verification.	Designing SuspiciousFlag, VerificationMethod, and the GPS constraint logic.
07	Write SMART objectives for a hostel management system project.	Drafting Section 1.3 specific objectives.
08	How to implement geofencing in a mobile app for student location verification?	Designing the GeofenceBoundary and GeofencePolygonPoints entities.
09	Convert an ERD into a normalized 3NF relational schema with functional dependencies explained.	Drafting Sections 3.1, 3.2, and 3.3.
10	Explain 1NF, 2NF, and 3NF with examples relevant to a student hostel database.	Expanding normalization analysis with SHMS-specific examples.
11	Create a stored procedures documentation table for a MySQL hostel management system.	Drafting the Section 4.3 procedures/triggers/views table.
12	What business rules should govern room allocation, exit applications, and attendance in a hostel system?	Developing the 18 business rules in Section 2.3.

Dates of Access: March 3, 2026 – June 13, 2026.